Queries and SubQueries

==========

What is Subquery

==========

a subquery is a query inside another query.

it is written inside parenthesis() and its result is used by the outer query.

it is also called.

inner Query

Nested Query

inner select.

general syntax

----

select column-list

from table

where column OPERATOR(SELECT COLUMN FROM TABLE WHERE CONDITION)

purpose of subquery

----

to break down complex problems into smaller queries.

to use the result of one query as input to another

to avoid multiple steps(write in a single Query insted of temp tables)

types of subqueries

----

1. Single-row-subquery

2. multi-row subquery

3. nested subquery.

4. Correlated subquery.

5. Exists/not Exists subquery

1. Single-row-subquery

----

1. Single Row Subqueries

Single row subqueries return exactly one row and one column. They use comparison operators like =, >, <, >=, <=, !=.

Characteristics:

- **Returns only one value**
- **Can use single-value comparison operators**
- **If subquery returns no rows or multiple rows, it causes an error**

Syntax:

SELECT columns

FROM table

WHERE column operator (SELECT column FROM table WHERE condition);

2. Multi-row Subquery (IN)

—----------

1. Multi-Row Subqueries with IN

A Multi-Row Subquery returns more than one row. The IN operator is used when you want to compare a value with multiple values returned by the subquery.

Characteristics

- **Returns multiple rows**
- **Uses the IN operator**
- **Checks whether a value exists in the list returned by the subquery**
- **Equivalent to multiple OR conditions**

- **Most commonly used for matching values**

Syntax

SELECT columns

FROM table

WHERE column IN

(SELECT column

FROM table

WHERE condition);

3. Multi-row Subquery (ANY / SOME)

—----------

Multi row subqueries return multiple rows. When using ANY or SOME (they are synonymous), the condition is true if it matches at least one value returned by the subquery.

Characteristics:

- **Returns multiple values**
- **ANY and SOME are interchangeable**
- **Condition is true if ANY of the values satisfy the condition**
- **Often used with IN, >ANY, <ANY, =ANY, etc.**

Syntax:

SELECT columns

FROM table

WHERE column operator ANY/SOME (SELECT column FROM table WHERE condition);

4. Multi-row Subquery (ALL)

—----------

The ALL operator means the condition must be true for ALL values returned by the subquery.

Characteristics:

- **Condition must be true for ALL values in the subquery result**
- **Often used with >ALL, <ALL, >=ALL, <=ALL**
- **>ALL means greater than the maximum value**
- **<ALL means less than the minimum value**

Syntax:

SELECT columns

FROM table

WHERE column operator ALL (SELECT column FROM table WHERE condition);

5. Nested Subquery

—----------

Nested subqueries are subqueries within subqueries. They can be multiple levels deep and are executed from the innermost to the outermost.

Characteristics:

- **Multiple levels of nesting**
- **Inner queries execute first**
- **Can combine different types of subqueries**
- **Useful for complex conditions**

Syntax:

```
SELECT column_list
FROM table
WHERE column = (SELECT column
        FROM table
        WHERE column = (SELECT column
                FROM table
                WHERE condition));
```

6. Correlated Subquery

—----------

Correlated subqueries are subqueries that reference columns from the outer query. Unlike regular subqueries that execute once, correlated subqueries execute once for each row processed by the outer query.

Characteristics:

- **References columns from the outer query**
- **Executes multiple times (once per outer row)**
- **Cannot be executed independently**
- **Often used with EXISTS, NOT EXISTS**
- **Generally slower than non-correlated subqueries**

Syntax

```
SELECT columns
FROM table1 outer_alias
WHERE condition (SELECT column
        FROM table2 inner_alias
    WHERE inner_alias.column = outer_alias.column);
```

7. EXISTS / NOT EXISTS Subquery

—----------

EXISTS and NOT EXISTS

EXISTS is used to test whether a subquery returns any rows. It returns TRUE if the subquery returns at least one row, FALSE otherwise. NOT EXISTS returns TRUE if the subquery returns no rows.

Characteristics:

- **Returns TRUE/FALSE (boolean result)**
- **More efficient than IN for large datasets**
- **Often used with correlated subqueries**
- **Stops execution as soon as first match is found**

- **SELECT clause in EXISTS doesn't matter (often use SELECT 1)**

Syntax:

SELECT columns

FROM table1

WHERE EXISTS (SELECT 1 FROM table2 WHERE condition);

SELECT columns

FROM table1

WHERE NOT EXISTS (SELECT 1 FROM table2 WHERE condition);

1. Sample Database

Table: Customers

customer_id	customer_name	city	customer_type
1	Rahul	Delhi	Premium
2	Meena	Mumbai	Regular
3	Arjun	Bangalore	Premium
4	Kavya	Delhi	Regular
5	Suresh	Chennai	Premium
6	Anjali	Mumbai	Regular
7	Vikram	Delhi	Regular
8	Pooja	Bangalore	Premium
9	Devi	Chennai	Regular

Table: Products

product_id	product_name	category	price
1	Laptop	Electronics	55000
2	Mobile	Electronics	25000
3	Chair	Furniture	7000
4	Table	Furniture	9000
5	Headphones	Electronics	3000
6	Bookshelf	Furniture	6000

Table: Orders

order_id	customer_id	product_id	quantity	order_date	amount
1	1	1	1	2023-01-05	55000
2	2	2	1	2023-01-10	25000
3	3	3	2	2023-02-01	14000
4	1	5	2	2023-02-15	6000
5	4	4	1	2023-03-03	9000
6	5	2	2	2023-03-20	50000
7	6	4	3	2023-04-02	18000
8	3	5	1	2023-04-15	3000
9	7	3	1	2023-05-01	7000
10	8	1	1	2023-05-20	54000
11	2	3	2	2023-06-02	12000
12	5	4	1	2023-06-15	8500

Note: "amount" is the total paid for that order (it may reflect quantity, discounts, etc., and does not need to equal price × quantity exactly).

2. Database Setup — CREATE TABLE & INSERT Statements

Run the script below in your SQL client (MySQL / SQL Server / Oracle / PostgreSQL — syntax may need minor tweaks) to build the exact database used throughout this worksheet.

Step 1: Create the tables

```
CREATE TABLE Customers (
  customer_id  INT PRIMARY KEY,
  customer_name  VARCHAR(50),
  city           VARCHAR(50),
  customer_type  VARCHAR(20)
);

CREATE TABLE Products (
  product_id    INT PRIMARY KEY,
  product_name   VARCHAR(50),
  category       VARCHAR(30),
  price          DECIMAL(10,2)
);

CREATE TABLE Orders (
  order_id       INT PRIMARY KEY,
  customer_id  INT REFERENCES Customers(customer_id),
  product_id    INT REFERENCES Products(product_id),
  quantity       INT,
```

```
  order_date    DATE,
  amount        DECIMAL(10,2)
);
```

Step 2: Insert Customers records

```
INSERT INTO Customers (customer_id, customer_name, city, customer_type) VALUES (1, 'Rahul', 'Delhi', 'Premium');

INSERT INTO Customers (customer_id, customer_name, city, customer_type) VALUES (2, 'Meena', 'Mumbai', 'Regular');

INSERT INTO Customers (customer_id, customer_name, city, customer_type) VALUES (3, 'Arjun', 'Bangalore', 'Premium');

INSERT INTO Customers (customer_id, customer_name, city, customer_type) VALUES (4, 'Kavya', 'Delhi', 'Regular');

INSERT INTO Customers (customer_id, customer_name, city, customer_type) VALUES (5, 'Suresh', 'Chennai', 'Premium');

INSERT INTO Customers (customer_id, customer_name, city, customer_type) VALUES (6, 'Anjali', 'Mumbai', 'Regular');

INSERT INTO Customers (customer_id, customer_name, city, customer_type) VALUES (7, 'Vikram', 'Delhi', 'Regular');

INSERT INTO Customers (customer_id, customer_name, city, customer_type) VALUES (8, 'Pooja', 'Bangalore', 'Premium');
```

```
INSERT INTO Customers (customer_id, customer_name, city, customer_type) VALUES (9, 'Devi', 'Chennai', 'Regular');
```

Step 3: Insert Products records

```
INSERT INTO Products (product_id, product_name, category, price) VALUES (1, 'Laptop', 'Electronics', 55000);

INSERT INTO Products (product_id, product_name, category, price) VALUES (2, 'Mobile', 'Electronics', 25000);

INSERT INTO Products (product_id, product_name, category, price) VALUES (3, 'Chair', 'Furniture', 7000);

INSERT INTO Products (product_id, product_name, category, price) VALUES (4, 'Table', 'Furniture', 9000);

INSERT INTO Products (product_id, product_name, category, price) VALUES (5, 'Headphones', 'Electronics', 3000);

INSERT INTO Products (product_id, product_name, category, price) VALUES (6, 'Bookshelf', 'Furniture', 6000);
```

Step 4: Insert Orders records

```
INSERT INTO Orders (order_id, customer_id, product_id, quantity, order_date, amount)
VALUES (1, 1, 1, 1, '2023-01-05', 55000);

INSERT INTO Orders (order_id, customer_id, product_id, quantity, order_date, amount)
VALUES (2, 2, 2, 1, '2023-01-10', 25000);

INSERT INTO Orders (order_id, customer_id, product_id, quantity, order_date, amount)
VALUES (3, 3, 3, 2, '2023-02-01', 14000);

INSERT INTO Orders (order_id, customer_id, product_id, quantity, order_date, amount)
VALUES (4, 1, 5, 2, '2023-02-15', 6000);

INSERT INTO Orders (order_id, customer_id, product_id, quantity, order_date, amount)
VALUES (5, 4, 4, 1, '2023-03-03', 9000);

INSERT INTO Orders (order_id, customer_id, product_id, quantity, order_date, amount)
VALUES (6, 5, 2, 2, '2023-03-20', 50000);

INSERT INTO Orders (order_id, customer_id, product_id, quantity, order_date, amount)
VALUES (7, 6, 4, 3, '2023-04-02', 18000);

INSERT INTO Orders (order_id, customer_id, product_id, quantity, order_date, amount)
VALUES (8, 3, 5, 1, '2023-04-15', 3000);

INSERT INTO Orders (order_id, customer_id, product_id, quantity, order_date, amount)
VALUES (9, 7, 3, 1, '2023-05-01', 7000);

INSERT INTO Orders (order_id, customer_id, product_id, quantity, order_date, amount)
VALUES (10, 8, 1, 1, '2023-05-20', 54000);

INSERT INTO Orders (order_id, customer_id, product_id, quantity, order_date, amount)
VALUES (11, 2, 3, 2, '2023-06-02', 12000);

INSERT INTO Orders (order_id, customer_id, product_id, quantity, order_date, amount)
VALUES (12, 5, 4, 1, '2023-06-15', 8500);
```

3. Single-Row Subqueries

A single-row subquery returns exactly one row and one column, so it is used with a plain comparison operator: =, >, <, >=, <=, or <>.

Q1. Find the name and price of the most expensive product.

Answer:

```
SELECT product_name, price
FROM Products
WHERE price = (SELECT MAX(price) FROM Products);
```

Result: *Laptop | 55000 (1 row)*

Q2. Find the order (order_id, customer_id, amount) with the smallest order amount.

Answer:

```
SELECT order_id, customer_id, amount
FROM Orders
WHERE amount = (SELECT MIN(amount) FROM Orders);
```

Result: *order_id = 8, customer_id = 3, amount = 3000 (1 row)*

Q3. Find the name of the customer who placed order number 5.

Answer:

```
SELECT customer_name
FROM Customers
WHERE customer_id = (SELECT customer_id FROM Orders WHERE order_id = 5);
```

Result: *Kavya (1 row)*

Q4. Find the category of the product that was purchased in order number 1.

Answer:

```
SELECT category
FROM Products
WHERE product_id = (SELECT product_id FROM Orders WHERE order_id = 1);
```

Result: *Electronics (1 row)*

Q5. Find the name of the product involved in the order with the highest amount.

Answer:

```
SELECT product_name
FROM Products
WHERE product_id = (
  SELECT product_id FROM Orders WHERE amount = (SELECT MAX(amount) FROM Orders)
);
```

Result: *Laptop (1 row)*

4. Multi-Row Subqueries — IN

The IN operator checks whether a value matches any value in the list returned by the subquery. NOT IN checks that it matches none of them.

Q1. Find the names of customers who live in 'Delhi' or 'Mumbai'.

Answer:

```
SELECT customer_name
FROM Customers
WHERE city IN ('Delhi', 'Mumbai');
```

Result: *Rahul, Meena, Kavya, Anjali, Vikram (5 rows)*

Q2. Find the names of products that have been ordered at least once.

Answer:

```
SELECT product_name
FROM Products
WHERE product_id IN (SELECT product_id FROM Orders);
```

Result: *Laptop, Mobile, Chair, Table, Headphones (5 rows)*

Q3. Find the names of customers who have placed at least one order.

Answer:

```
SELECT customer_name
FROM Customers
WHERE customer_id IN (SELECT customer_id FROM Orders);
```

Result: *All customers except Devi (8 rows)*

Q4. Find products that have never been ordered (use NOT IN).

Answer:

```
SELECT product_name
FROM Products
WHERE product_id NOT IN (SELECT product_id FROM Orders);
```

Result: *Bookshelf (1 row)*

Q5. Find customers whose city matches a city where at least one 'Premium' customer lives.

Answer:

```
SELECT customer_name
FROM Customers
WHERE city IN (SELECT city FROM Customers WHERE customer_type = 'Premium');
```

Result: *Rahul, Arjun, Kavya, Suresh, Vikram, Pooja, Devi (7 rows)*

5. Multi-Row Subqueries — ANY / SOME

ANY (and its synonym SOME) makes the outer comparison true if it holds for at least one value returned by the inner query. For example, "> ANY" means greater than the smallest value returned.

Q1. Find products priced greater than ANY product in the 'Furniture' category.

Answer:

```
SELECT product_name, price
FROM Products
WHERE price > ANY (SELECT price FROM Products WHERE category = 'Furniture');
```

Result: *Laptop, Mobile, Chair, Table (4 rows)*

Q2. Find orders with an amount greater than ANY order placed by 'Arjun'.

Answer:

```
SELECT order_id, amount
FROM Orders
WHERE amount > ANY (
  SELECT amount FROM Orders
  WHERE customer_id = (SELECT customer_id FROM Customers WHERE customer_name = 'Arjun')
);
```

Result: *All orders except order_id 8 (11 rows)*

Q3. Find products priced less than SOME product in the 'Electronics' category.

Answer:

```
SELECT product_name, price
FROM Products
WHERE price < SOME (SELECT price FROM Products WHERE category = 'Electronics');
```

Result: *Mobile, Chair, Table, Headphones, Bookshelf (5 rows)*

Q4. Find orders with an amount greater than ANY order placed by a 'Regular' customer.

Answer:

```
SELECT order_id, amount
FROM Orders
WHERE amount > ANY (
  SELECT o.amount FROM Orders o
  JOIN Customers c ON o.customer_id = c.customer_id
  WHERE c.customer_type = 'Regular'
);
```

Result: *All orders except order_id 4, 8, 9 (9 rows)*

Q5. Find orders with an amount less than ANY order placed by a customer from 'Mumbai'.

Answer:

```
SELECT order_id, amount
FROM Orders
WHERE amount < ANY (
  SELECT o.amount FROM Orders o
  JOIN Customers c ON o.customer_id = c.customer_id
  WHERE c.city = 'Mumbai'
);
```

Result: *order_id 3, 4, 5, 7, 8, 9, 11, 12 (8 rows)*

6. Multi-Row Subqueries — ALL

ALL makes the outer comparison true only if it holds against every value returned by the inner query. For example, "> ALL" means greater than the largest value returned, and "< ALL" means less than the smallest value returned.

Q1. Find products priced higher than ALL products in the 'Furniture' category.

Answer:

```
SELECT product_name, price
FROM Products
WHERE price > ALL (SELECT price FROM Products WHERE category = 'Furniture');
```

Result: *Laptop, Mobile (2 rows)*

Q2. Find orders with an amount greater than ALL orders placed by 'Meena'.

Answer:

```
SELECT order_id, amount
FROM Orders
WHERE amount > ALL (
  SELECT amount FROM Orders
  WHERE customer_id = (SELECT customer_id FROM Customers WHERE customer_name = 'Meena')
);
```

Result: *order_id 1, 6, 10 (3 rows)*

Q3. Find products priced lower than ALL products in the 'Furniture' category.

Answer:

```
SELECT product_name, price
FROM Products
WHERE price < ALL (SELECT price FROM Products WHERE category = 'Furniture');
```

Result: *Headphones (1 row)*

Q4. Find orders with an amount greater than ALL orders placed for 'Furniture' category products.

Answer:

```
SELECT o.order_id, o.amount
FROM Orders o
WHERE o.amount > ALL (
  SELECT o2.amount FROM Orders o2
  JOIN Products p ON o2.product_id = p.product_id
  WHERE p.category = 'Furniture'
);
```

Result: *order_id 1, 2, 6, 10 (4 rows)*

Q5. Find orders with an amount less than ALL orders placed for 'Furniture' category products.

Answer:

```
SELECT o.order_id, o.amount
FROM Orders o
WHERE o.amount < ALL (
  SELECT o2.amount FROM Orders o2
  JOIN Products p ON o2.product_id = p.product_id
  WHERE p.category = 'Furniture'
);
```

Result: *order_id 4, 8 (2 rows)*

7. Nested Subqueries

A nested subquery is a subquery placed inside another subquery. SQL evaluates from the innermost query outward, and each result feeds the query one level up.

Q1. Find the name(s) of the customer(s) who ordered the most expensive product.

Answer:

```
SELECT customer_name
FROM Customers
WHERE customer_id IN (
  SELECT customer_id FROM Orders
  WHERE product_id = (
          SELECT product_id FROM Products
          WHERE price = (SELECT MAX(price) FROM Products)
  )
);
```

Result: *Rahul, Pooja (2 rows — both bought the Laptop)*

Q2. Find the category of the product bought in the highest-amount order.

Answer:

```
SELECT category
FROM Products
WHERE product_id = (
  SELECT product_id FROM Orders
  WHERE amount = (SELECT MAX(amount) FROM Orders)
);
```

Result: *Electronics (1 row)*

Q3. Find the city (or cities) of the customer(s) who bought the cheapest product.

Answer:

```
SELECT city
FROM Customers
WHERE customer_id IN (
  SELECT customer_id FROM Orders
  WHERE product_id = (
          SELECT product_id FROM Products
          WHERE price = (SELECT MIN(price) FROM Products)
  )
);
```

Result: *Delhi, Bangalore (2 rows — Rahul and Arjun bought the Headphones)*

Q4. Find the names of customers who live in the same city as the customer who placed the highest-amount order.

Answer:

```
SELECT customer_name
FROM Customers
WHERE city = (
  SELECT city FROM Customers
  WHERE customer_id = (
          SELECT customer_id FROM Orders
          WHERE amount = (SELECT MAX(amount) FROM Orders)
  )
);
```

Result: *Rahul, Kavya, Vikram (3 rows — all live in Delhi)*

Q5. Find the name(s) of products ordered by 'Rahul' that cost more than the average price of all products.

Answer:

```
SELECT product_name
FROM Products
WHERE product_id IN (
  SELECT product_id FROM Orders
  WHERE customer_id = (SELECT customer_id FROM Customers WHERE customer_name = 'Rahul')
)
AND price > (SELECT AVG(price) FROM Products);
```

Result: *Laptop (1 row; average product price is 17,500)*

8. Correlated Subqueries

A correlated subquery references a column from the outer query, so it cannot run on its own — it is re-evaluated once for every row the outer query processes.

Q1. Find customers who have placed more than one order.

Answer:

```
SELECT customer_name
FROM Customers c
WHERE (SELECT COUNT(*) FROM Orders o WHERE o.customer_id = c.customer_id) > 1;
```

Result: *Rahul, Meena, Arjun, Suresh (4 rows)*

Q2. Find the highest-amount order placed by each customer.

Answer:

```
SELECT o1.order_id, o1.customer_id, o1.amount
FROM Orders o1
WHERE o1.amount = (
```

```
  SELECT MAX(o2.amount) FROM Orders o2
  WHERE o2.customer_id = o1.customer_id
);
```

Result: *order_id 1, 2, 3, 5, 6, 7, 9, 10 (8 rows — one top order per customer)*

Q3. Find products priced higher than the average price of products in their own category.

Answer:

```
SELECT p1.product_name, p1.category, p1.price
FROM Products p1
WHERE p1.price > (
  SELECT AVG(p2.price) FROM Products p2
  WHERE p2.category = p1.category
);
```

Result: *Laptop (Electronics), Table (Furniture) (2 rows)*

Q4. Find customers whose order amount is greater than the average order amount of all customers from their own city.

Answer:

```
SELECT DISTINCT c1.customer_name
FROM Customers c1
JOIN Orders o1 ON c1.customer_id = o1.customer_id
WHERE o1.amount > (
  SELECT AVG(o2.amount) FROM Orders o2
  JOIN Customers c2 ON o2.customer_id = c2.customer_id
  WHERE c2.city = c1.city
);
```

Result: *Rahul, Meena, Pooja, Suresh (4 rows)*

Q5. Find the second-lowest priced product within each category.

Answer:

```
SELECT p1.product_name, p1.category, p1.price
FROM Products p1
WHERE 1 = (
  SELECT COUNT(*) FROM Products p2
  WHERE p2.category = p1.category AND p2.price < p1.price
);
```

Result: *Mobile (Electronics, 25000), Chair (Furniture, 7000) (2 rows)*

9. EXISTS and NOT EXISTS

EXISTS only checks whether the subquery returns any rows at all (true/false) — the actual values don't matter. NOT EXISTS is true when the subquery returns no rows.

Q1. Find customers who have placed at least one order (use EXISTS).

Answer:

```
SELECT customer_name
FROM Customers c
WHERE EXISTS (SELECT 1 FROM Orders o WHERE o.customer_id = c.customer_id);
```

Result: *All customers except Devi (8 rows)*

Q2. Find customers who have never placed an order (use NOT EXISTS).

Answer:

```
SELECT customer_name
```

```
FROM Customers c
WHERE NOT EXISTS (SELECT 1 FROM Orders o WHERE o.customer_id = c.customer_id);
```

Result: *Devi (1 row)*

Q3. Find products that have been ordered at least once (use EXISTS).

Answer:

```
SELECT product_name
FROM Products p
WHERE EXISTS (SELECT 1 FROM Orders o WHERE o.product_id = p.product_id);
```

Result: *Laptop, Mobile, Chair, Table, Headphones (5 rows)*

Q4. Find products that have never been ordered (use NOT EXISTS).

Answer:

```
SELECT product_name
FROM Products p
WHERE NOT EXISTS (SELECT 1 FROM Orders o WHERE o.product_id = p.product_id);
```

Result: *Bookshelf (1 row)*

Q5. Find customers who have not ordered any product from the 'Electronics' category (use NOT EXISTS).

Answer:

```
SELECT customer_name
FROM Customers c
WHERE NOT EXISTS (
  SELECT 1 FROM Orders o
  JOIN Products p ON o.product_id = p.product_id
```

```
  WHERE o.customer_id = c.customer_id AND p.category = 'Electronics'
);
```

Result: *Kavya, Anjali, Vikram, Devi (4 rows)*